

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ЭВМ

Отчет по лабораторной работе № 2
Прерывания. Таймеры
Вариант №9

Выполнил:
студенты группы 250501 Снитко Д.А.
Герасимович В.С.

Проверил:
ассистент каф. ЭВМ _____ Шеменков В.В.

Минск 2025

1 ЦЕЛЬ РАБОТЫ

Ознакомиться с работой подсистемы прерываний и таймерами микроконтроллера MSP430F5529.

2 ИСХОДНЫЕ ДАННЫЕ

В соответствии с вариантом, используя прерывания и таймеры, запрограммировать кнопки и светодиоды. Для работы с кнопками использовать только прерывания. Не использовать опросы флагов состояния в цикле и циклы задержки (активное ожидание). Не допускается использовать иные заголовочные файлы, кроме `msp430`, не допускается также использовать высокоуровневые библиотеки. При выполнении задания особое внимание уделить грамотному выбору режима работы таймера. Комментарии в тексте программы обязательны, они должны пояснять что именно делает данный фрагмент.

Задание по варианту:

Диоды, кнопки и таймеры: LED4 – LED8, S1, TA0. Режим: перемещение. Переключение: включение и выключение режима нажатием кнопки. Диод на некоторое время включается, потом гаснет, и включается следующий.

3 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1 Прерывания

Различают системные немаскируемые (SMNI), пользовательские немаскируемые (UNMI) и маскируемые прерывания. К системным немаскируемым относятся: сигнал RST/NMI в режиме NMI, сбой генератора, ошибка доступа Flash памяти. К пользовательским немаскируемым – сбой напряжения питания (от подсистемы PMM), доступ к несуществующей (vacant) памяти, события с буфером (mailslot) JTAG интерфейса. Маскируемые прерывания могут быть отключены (замаскированы) индивидуально или все сразу (бит GIE регистра состояния SR).

Кратко рассмотрим, как происходит обработка прерывания. Задержка от возникновения запроса на прерывание до начала выполнения обработчика составляет 6 циклов. При этом заканчивается выполнение текущей инструкции, счетчик команд PC сохраняется в стеке (указывает на следующую команду), регистр состояния SR сохраняется в стеке, выбирается прерывание с максимальным приоритетом (если поступило несколько запросов), автоматически сбрасывается флаг запроса от отдельного прерывания (сброс общего флага запроса должен осуществляться программно). Далее, все биты SR сбрасываются, за исключением SCG0, так как останавливаются все режимы с низким питанием. Так как бит GIO при этом устанавливается в 0, все прерывания запрещаются. Наконец, вектор (адрес обработчика) загружается в PC.

Из-за конвейерной архитектуры процессора, команда, следующая за EINT (разрешение прерывания), всегда выполняется, даже если запрос на прерывание возник до его разрешения. Если за EINT сразу следует DINT, прерывание, ожидающее обработки может быть не обслужено. Команды, следующие за DINT, в этом случае могут сработать некорректно. Аналогичные последствия вызываются альтернативными командами, которые устанавливают и сразу сбрасывают флаг GIE регистра состояний. Рекомендуется вставлять хотя бы одну команду между EINT и DINT. Возврат из прерывания выполняется командой RETI, которая выполняется за 5 циклов и загружает из стека SR, PC. Таблица векторов прерываний располагается по адресам 0FFFFh – 0FF80h и содержит 64 вектора. Бит SYSRIVECT регистра SYSCTL позволяет определить альтернативную таблицу векторов, в старших адресах RAM. По сигналу сброса этот бит автоматически сбрасывается. За прерывания отвечают ряд системных регистров.

Пользовательские маскируемые прерывания рассматриваются отдельно при обсуждении соответствующего функционального узла архитектуры микроконтроллера, в частности, ранее уже рассматривались регистры для работы с прерываниями от цифровых портов ввода-вывода.

Работа с прерываниями достаточно проста. Вначале необходимо разрешить соответствующее прерывание, например, `P1IE |= BIT7`; - разрешает прерывание по входу 7 вывода порта 1, в экспериментальной плате к нему подключена кнопка S1. После того, как режим инициализирован, хорошим тоном считается перевод контроллера в режим пониженного энергопотребления. Сделать это можно, используя вызов `__bis_SR_register`, например, следующий фрагмент переводит контроллер в режим LPM0 с разрешением прерываний: `__bis_SR_register(LPM0_bits + GIE)`;

Еще одной особенностью запуска в среде отладки Code Composer Studio является необходимость вызова `__no_operation()` перед завершением функции `main`, если она не использует некоторого цикла. Без этого вызова с завершением функции `main` завершится и выполнение кода в оболочке.

Собственно обработчик прерывания описывается с использованием директивы `#pragma vector`. Например, фрагмент кода ниже описывает обработчик прерывания от порта ввода-вывода 1:

```
#pragma vector=PORT1_VECTOR
__interrupt void PORT1_ISR(void)
```

3.2 Таймеры

MSP430F5529 содержит 32-разрядный сторожевой таймер WDT (базовый адрес 015Ch), 3 таймера TAх (базовые адреса соответственно 0340h, 0380h, 0400h), таймер TBх (базовый адрес 03C0h) и таймер часов реального времени RTC_A (базовый адрес 04A0h).

Основная функция сторожевого таймера WDT – генерация сигнала сброса при программном сбросе, например, зацикливании: если заданный интервал времени истек, генерируется сигнал сброса. WDT может быть сконфигурирован как интервальный и генерировать сигналы прерываний по истечении заданного промежутка времени.

Таймер А – это 16-разрядный таймер/счетчик с широкими возможностями по использованию прерываний, которые могут генерироваться счетчиком в случае переполнения и от каждого регистра захвата/сравнения.

Таймер А обладает следующими возможностями:

- асинхронный 16-битный таймер/счетчик с четырьмя рабочими режимами;
- выбираемый и конфигурируемый источник счетного импульса;
- три конфигурируемых регистра захвата/сравнения (в таймере TA0 их 5);
- возможность множественного захвата/сравнения;
- конфигурируемые выводы с возможностью широко-импульсной модуляции;

- асинхронная фиксация (защелка) входа и выхода;
- счет по фронту тактового импульса;
- возможность генерации прерываний при переполнении;
- регистр вектора прерываний для быстрого декодирования всех прерываний таймера А.

Источниками входного импульса для таймера А могут быть следующие тактовые сигналы: ACLK, SMCLK, внешние CAxCLK, INCLK. На входе имеется программно доступный делитель частоты, который позволяет снижать частоту в 2,3,4,5,6,7,8 раз. Режимы работы таймера: остановка, прямой счет (до уровня TAxCCR0) (Up Mode), непрерывный режим (Continuous Mode), реверсивный счет (Up/Down mode).

Таймер В имеет ряд отличий от таймера А:

- 7 регистров захвата/сравнения;
- разрядность счетчика программируется (8, 10, 12, 16 бит);
- регистр TBxCCRn с двойной буферизацией и может быть сгруппирован;
- все выходы имеют высокоимпедансное состояние;
- не поддерживается бит SCCI.

Таймер часов реального времени RTC_A представляет собой конфигурируемые часы реального времени с функцией календаря и счетчика общего назначения. Поддерживает выбор формата BCD или двоичный в режиме часов реального времени, имеет программируемый будильник, подстройку коррекции времени, возможность прерываний. Рассмотрим подробнее работу со сторожевым таймером WDT. Он имеет 8 программно выбираемых временных интервалов, поддерживает сторожевой и интервальный режимы, обеспечивает защиту доступа к управляющему регистру, может отключаться для экономии энергии. Важным свойством WDT является отказоустойчивый сигнал (источник счетного сигнала не может быть отключен, пока таймер в сторожевом режиме). Это может не позволить перейти в режим пониженного потребления энергии (LPM).

Регистр счетчика WDT непосредственно программно не доступен. Сигнал на счетный вход может подаваться с тактовых линий SMCLK, ACLK, VLOCLK либо X_CLK от некоторых устройств. После сброса сторожевой таймер настроен на сторожевой режим, входным выбран сигнал от SMCLK. Поэтому необходимо остановить, установить либо сбросить таймер до истечения установленного интервала, иначе будет сгенерирован сигнал сброса PUC. Флаг запроса на прерывание сбрасывается автоматически после обслуживания, также может быть сброшен программно. Адреса обработчиков в сторожевом и интервальном режиме различны. Разрешение прерываний WDT осуществляется

битом WDTIE регистра SFR1E1, флаг прерывания — бит WDTIFG в регистре SFR1FG1.

Рассмотрим особенности работы с таймером А, таймеры В и RTC_A подробно рассматривать не будем. Структура таймера изображена на рис. 2.1. В последующих таблицах представлены регистры таймера и некоторые поля регистров таймера.

Первым этапом выполняется инициализация таймера TAx с помощью регистров TAxCTL, TAxCCRn и TAxCCTLn. В регистре TAxCTL рекомендуется выбрать в качестве источника тактирования SMCLK с выходной частотой тактирования 1МГц, режим счета, коэффициент деления и установить бит TACLR. В регистре счета/сравнения TAxCCTLn необходимо разрешить прерывания. В 16-битном регистре TAxCCRn указывается значение счетчика, при достижении которого в режиме прямого или реверсивного счета генерируется прерывание. При захвате значения для его сохранения также используется данный регистр. Сброс состояния таймера осуществляется путем записи нулевого значения в конфигурационные регистры. TAxIV – 16-разрядный регистр вектора прерывания. Биты 0-2 регистра TAxEX0 (поле TAIDEX) устанавливают параметры расширенного делителя входа (от деления на 1 при 000b до деления на 8 при 111b).

В режиме прямого счета таймер считает от 0 до значения, установленного в регистре TAxCCR0. При достижении установленного значения таймер продолжает счет с 0. Количество тактовых импульсов в периоде равно TAxCCR0+1. Флаг прерывания TAxCCR0 CCIFG устанавливается, когда счетчик досчитал до значения TAxCCR0. Флаг прерывания TAxCTL TAIFG устанавливается, когда счетчик переходит от TAxCCR0 к 0.

В непрерывном режиме таймер считает от 0 до 0xFFFFh. Регистр захвата/сравнения TAxCCR0 работает аналогично остальным регистрам захвата/сравнения. Флаг прерывания TAxCTL TAIFG устанавливается, когда счетчик переходит от 0xFFFFh к 0.

Непрерывный режим можно использовать для генерирования независимых выходных интервалов и временных частот. При окончании любого из интервалов, генерируется прерывание. Следующий временной интервал добавляется к TAxCCRn обработчиком прерываний.

В реверсивном режиме таймер считает от 0 до значения, установленного в регистре TAxCCR0. При достижении установленного значения таймер продолжает счет в обратном направлении к 0. Период счета равен удвоенному значению TAxCCR0. Направление счета запоминается, что позволяет выполнять остановку таймера, а затем продолжить счет с прерванного места. Флаг прерывания TAxCCR0 CCIFG устанавливается, когда счетчик досчитал до значения TAxCCR0. Флаг прерывания TAxCTL TAIFG устанавливается, когда счетчик досчитал в обратном направлении от TAxCCR0 до 0.

Реверсивный режим позволяет поддерживать пустые интервалы (Dead Time) между выходными сигналами, когда ни один из них не активен. Регистры TAxCCRn не имеют буфера, поэтому они изменяются сразу после записи.

Режим захвата выбирается, когда CUP = 1. Используется для записи времени какого-либо события. В качестве сигналов на входы захвата CCIxA и CCIxB могут быть поданы сигналы с внешних выводов или внутренние сигналы. Источник выбирается CCIS битами. Биты CM определяют, будет ли захват происходить по фронту сигнала, по спаду, либо и по фронту, и по спаду. При наступлении соответствующего события значение счетчика копируется в регистр TAxCCRn и устанавливается флаг прерываний CCIFG. Уровень входного сигнала может быть прочитан в любое время из бита CCI. Поскольку сигнал на входе не синхронизирован с тактовыми импульсами, могут возникать гонки. Поэтому рекомендуется устанавливать бит SCS, чтобы захват происходил с началом очередного тактового импульса. Захват может быть выполнен программно. Режим сравнения выбирается, когда CUP = 0. Используется для генерации на выходе ШИМ-сигнала или прерывания через заданный временной интервал. Когда счетчик достигает значения TAxCCRn, устанавливается флаг прерывания CCIFG, внутренний сигнал EQUn устанавливается в 1, EQUn влияет на выход в соответствии с режимом, а входной сигнал CCI защелкивается в регистре SCCI.

Каждый блок захвата/сравнения содержит выходной модуль, который формирует выходной сигнал на основе EQU0 и EQUn сигналов в зависимости от установленного режима выхода. Биты OUTMOD позволяют задать один из 8 режимов. Сигнал OUTn изменяется по переднему фронту синхросигнала, за исключением режима 0. Режимы 2, 3, 6 и 7 не пригодны для использования с выходным блоком 0, поскольку EQUn = EQU0.

Режимы выхода:

000 — Значение бита OUT. Сигнал OUTn изменяется сразу же с изменением бита OUT;

001 — Установка. Однократная установка при достижении заданного значения TAxCCRn;

010 — Переключение/сброс. Выход меняется при достижении значения TAxCCRn, сбрасывается при достижении TAxCCR0;

011 — Установка/сброс. Выход устанавливается при достижении значения TAxCCRn, сбрасывается при достижении TAxCCR0;

100 — Переключение. Выход меняется при достижении значения TAxCCRn;

101 — Сброс. Однократный сброс при достижении заданного значения TAxCCRn;

110 — Переключение/установка. Выход меняется при достижении значения TAxCCRn, устанавливается при достижении TAxCCR0;

111 — Сброс/установка. Выход сбрасывается при достижении значения TAxCCRn, устанавливается при достижении TAxCCR0.

При использовании констант (msp430f5529.h) не стоит забывать принципы их именования:

- константа, соответствующая биту поля-флага, именуется по имен поля,
- например, полю CPUOFF регистра состояния процессора SR (бит 4)
- соответствует константа CPUOFF;
- константа, соответствующая биту n в поле NNN, именуется NNNn;
- константа, соответствующая номеру x выбранного варианта для поля
- NNN именуется NNN_x;
- константа, соответствующая выбранному режиму zz для поля NNN
- именуется NNN__zz.

Так, например, для 3-битного поля SELA, константа, соответствующая 0 биту поля, именована SELA0, вариант выбора 0 (SELA = 000) именован SELA_0, а режим, соответствующий данному варианту именован SELA__XT1CLK. В некоторых случаях поля задают делители либо множители, соответствующие степени двойки. Тут надо быть особо внимательным и не спутать похожие мнемоники, например, NN4 (четвертый бит, т.е. 10000), NN_4(четвертый вариант, т.е. 00100), NN__4 (режим деления на 4, т.е. 00011).

4 ВЫПОЛНЕНИЕ РАБОТЫ

```
#include <msp430.h>

#define LED4 BIT1      // P1.1 - светодиод 4
#define LED5 BIT2      // P1.2 - светодиод 5
#define LED6 BIT3      // P1.3 - светодиод 6
#define LED7 BIT4      // P1.4 - светодиод 7
#define LED8 BIT5      // P1.5 - светодиод 8
#define S1 BIT7        // P1.7 - кнопка S1
#define ALL_LEDS (LED4 | LED5 | LED6 | LED7 | LED8) // Все
светодиоды

volatile unsigned char mode = 0;          // Режим: 0 - выкл, 1 -
бегущий свет
volatile unsigned char current_led = 0; // Текущий светодиод (0-4)
volatile unsigned char button_debounce = 0; // Флаг для
антидребезга

// Инициализация портов ввода-вывода
void init_GPIO(void) {
    P1DIR |= ALL_LEDS;          // Настройка светодиодов как выходы
    P1OUT &= ~ALL_LEDS;         // Погасить все светодиоды
    P1DIR &= ~S1;               // Кнопка S1 как вход
    P1REN |= S1;                // Включить подтягивающий резистор
    P1OUT |= S1;                // Подтяжка к VCC
    P1IES |= S1;                // Прерывание по спаду (нажатие
кнопки)
    P1IE |= S1;                 // Включить прерывание для S1
}

// Инициализация таймера TA0
void init_timer(void) {
    // Настройка TA0 в режиме Up с частотой ~2 Гц (500 мс)
    // Источник: SMCLK (1 МГц), делитель 8, период 62500
    TA0CTL = TASSEL_2 | ID_3 | MC_0; // SMCLK, делитель 8, таймер
остановлен
    TA0CCR0 = 62500;             // Период ~500 мс при 1 МГц/8
    TA0CCTL0 = CCIE;            // Включить прерывание по
совпадению
    TA0CTL |= TACLR;            // Сбросить таймер
}

// Основная функция
void main(void) {
    WDTCTL = WDTPW | WDTHOLD; // Отключить сторожевой таймер
    init_GPIO();              // Инициализация портов
    init_timer();             // Инициализация таймера
}
```

```

    __bis_SR_register(GIE);    // Разрешить глобальные прерывания
while (1) {
    // Вход в режим низкого энергопотребления LPM0
    __bis_SR_register(LPM0_bits | GIE);
}
}

// Обработчик прерывания для порта 1 (кнопка S1)
#pragma vector=PORT1_VECTOR
__interrupt void PORT1_ISR(void) {
    if (P1IFG & S1) {          // Проверка флага прерывания кнопки
        if (!button_debounce) { // Обработка только если нет
дребезга
            button_debounce = 1; // Установить флаг антидребезга
            mode = !mode;        // Переключить режим
            P1OUT &= ~ALL_LEDS;  // Погасить все светодиоды
            current_led = 0;      // Сбросить текущий светодиод
            if (mode) {
                TA0CTL |= MC_1;   // Запустить таймер в режиме Up
            } else {
                TA0CTL &= ~MC_3;  // Остановить таймер
            }
            // Короткая задержка для антидребезга (~10 мс)
            TA0CCR1 = 1250;       // 10 мс при SMCLK/8
            TA0CCTL1 = CCIE;      // Включить прерывание для
антидребезга
            TA0CTL |= MC_1;       // Запустить таймер даже если
mode = 0
        }
        P1IFG &= ~S1;            // Сбросить флаг прерывания
    }
}

// Обработчик прерывания таймера TA0
#pragma vector=TIMER0_A0_VECTOR
__interrupt void TIMER0_A0_ISR(void) {
    // Массив светодиодов для последовательного включения
    static const unsigned char leds[] = {LED4, LED5, LED6, LED7,
LED8};

    // Погасить все светодиоды
    P1OUT &= ~ALL_LEDS;

    if (mode) {
        // Включить текущий светодиод
        P1OUT |= leds[current_led];
        // Перейти к следующему светодиоду
        current_led = (current_led + 1) % 5;
    }
}

```

```

}

// Обработчик прерывания для TA0CCR1 (антидребезг)
#pragma vector=TIMER0_A1_VECTOR
__interrupt void TIMER0_A1_ISR(void) {
    if (TA0IV == TA0IV_TACCR1) { // Проверка источника прерывания
        button_debounce = 0;      // Сбросить флаг антидребезга
        TA0CCTL1 &= ~CCIE;        // Отключить прерывание CCR1
        if (!mode) {
            TA0CTL &= ~MC_3;      // Остановить таймер, если режим
выключен
        }
    }
}

```

5 ВЫВОДЫ

В ходе выполнения лабораторной работы была написана программа с использованием таймеров и прерываний в соответствии с заданием варианта.